

2^{do} PARCIAL ISW

TEST DRIVEN DEVELOPMENT

Los desarrolladores diseñan las pruebas unitarias ANTES de escribir código.
Control de calidad proactivo.

Precondiciones: establecen el estado inicial necesario para ejecutar correctamente los pasos de prueba.

Pasos del caso de prueba: reflejan interacciones del usuario con el sistema y realizan el llamado a la U.S.

Resultados: define qué debe devolver el sistema y valida que el comportamiento es correcto. (assert)

Mensaje final: si todas las verificaciones anteriores con assert son verdaderas, se imprimirá este mensaje, indicando que la prueba ha sido exitosa.

Beneficios del TDD

- Código robusto: más predecible y limpio, que permite saber cuándo está terminado. Es más tolerable al cambio, más seguro y reduce la duplicación de código. Es más barato de mantener.
- Acelera la velocidad de desarrollo y la posibilidad de despliegue continuo.
- Asegura cobertura de prueba → código de calidad.
- Obliga a pensar en cómo usar el componente.
- Promueve el desarrollo de componentes con alta cohesión y bajo acoplamiento.

El ♥ de TDD: Red-Green-Refactor

3 leyes de TDD

- no escribir una línea de código hasta que no hayas escrito antes un test case que falle.
- no es necesario escribir más test de los necesarios, 1 es suficiente.
- no vas a escribir más código en producción que el necesario por que el test pase.

Escribir un test

← Pasos del TDD

- ↳ Hacer que el test compile
- ↳ Hacer que el test falle (Red)
- ↳ Hacer un cambio por el que pase el test
- ↳ Correr el test por el que pase (Green)
- ↳ Refactoring

Patrón de TDD

Patrón Red Ber

- One Step Test
- Shorter Step Test
- Explanation Test
- Learning Test
- Another Test
- Regression Test

Patrón Green Ber

- Fake it
- Triangulation
- Obvious implementation
- One to many

Patrón de TDD

- Test
- Isolated Test
- Test List
- Test first
- Assert test
- Test data
- Evident data

Refactoring: forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos. Preserva la estructura externa, mejora la estructura interna.

Mejora la calidad de SW sin cambiar el comportamiento observable.

¿Por qué? Evita el deterioro del diseño, limpiar, simplificar, encontrar errores, incorporar aprendizaje sobre la aplicación.

Asegurar que las pruebas pasen

Encontrar un código que "huela mal"

Determinar cómo simplificar el código

Ejecutar pruebas por asegurarse que las pruebas siguen pasando

Hacer las simplificaciones

TESTING ÁGIL

Manifiesto del testing

- Probar durante todo el proceso, no al final
- Prevenir defectos sobre encontrarlos
- Entender que se está probando sobre verificar la funcionalidad
- Construir un mejor sist sobre romperlo
- Resp. del equipo entero, no solo el tester

9 principios para el testing en proyectos ágiles:

- Testing se mueve hacia adelante en el proyecto
- Testing no es una fase
- Todos hacen testing
- Reducir la latencia del feedback
- Las pruebas representan expectativas
- Mantener el código limpio, corregir defectos rápido
- Reducir la sobrecarga de documentación de las pruebas
- Las pruebas son parte del done
- De probar el final a conducido por pruebas

6 prácticas concretas

- Pruebas de unidad e integración automatizadas
- TDD
- ATDD
- Pruebas exploratorias
- Pruebas de regresión a nivel de sistema automatizadas
- Control de versión de las pruebas con el código

PRUEBAS ORIENTADAS AL NEGOCIO

Ordenamiento de Testing Ágil

GUÍAN EL DESARROLLO

Ejemplos, simulaciones,
prototipos, pruebas de ^{del historial} aceptación,
pruebas de UX

Pruebas unitarias, pruebas de
componentes (a nivel de código)

Pruebas exploratorias, flujos
de trabajo, pruebas de usabilidad,
pruebas de aceptación (UAT)

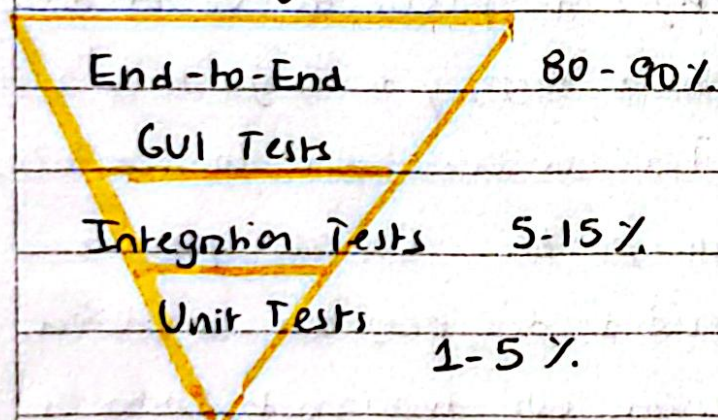
Pruebas de rendimiento, de carga,
de seguridad, recuperabilidad

CONTINUAN EL PRODUCTO

Pirámide de Testing Ágil

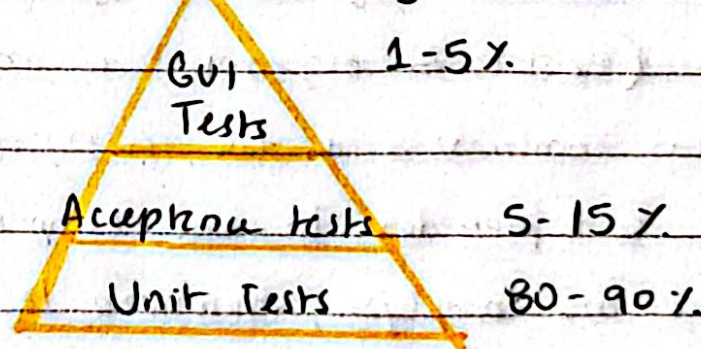


Tradicional (buscar bugs)



Ágil

(Prevenir Bugs)



Beneficios de la automatización del testing

- rápida ejecución
- mayor cobertura
- mayor alcance
- reusabilidad
- reducción de costos
- mayor precisión

TESTING

Proceso destructivo que trata de encontrar defectos en el código

Detectar errores en forma temprana ahorra esfuerzos, tiempo, recursos.

El testing no puede asegurar calidad. Esto depende de tareas realizadas durante todo el proceso.

Error vs Defecto

error: se encuentra en la etapa donde estamos trabajando

defecto: es un error que pasó por etapas sin ser detectado

Severidad

Prioridad

Bloqueante

Crítico

Mayor

Menor

Cosmético

Urgencia

Alta

Media

Baja

NIVELES DE TESTING

Unitario: Se prueba cada componente tras su realización. Se prueban componentes individuales, independientemente. Los errores suelen repararse tan pronto como se encuentran, sin constancia oficial de los incidentes. Se produce con acceso al código bajo pruebas y con el apoyo del entorno de desarrollo, tales como un framework de pruebas unitarias o herramienta de depuración.

De integración: Orientado a verificar que las partes de un sist que funcionan bien aisladamente, funcionan en conjunto. Cualquier estrategia de prueba de versión debe ser incremental (existen 2 esquemas, de arriba hacia abajo, y de abajo hacia arriba). Lo ideal es una combinación de los 2 esquemas. Tener en cuenta que los módulos críticos deben ser probados lo más tempranamente posible. Conectar poco a poco las partes más complejas, minimizar la necesidad de programas auxiliares.

De Sistema: prueba realizada cuando una aplicación está funcionando como un todo (pruebas de la construcción final). Trá de determinar si el sist es su globalidad opera satisfactoriamente (recuperación de fallas, seguridad y protección, performance, etc). Deben investigar tanto req funcionales y no funcionales del sist. El entorno de pruebas debe corresponder al entorno de producción tanto como sea posible.

De Aceptación: realizada por el usuario, para determinar si la aplicación se ajusta a sus necesidades. Meta: establecer confianza en el sistema, las partes del sistema o las características específicas y no funcionales. Encontrar defectos no es el foco principal. Comprende tanto la prueba realizada por el usuario en ambiente de laboratorio (α) como en ambiente de trabajo real (β).

Ambiente para la construcción de SW

- De desarrollo
- De prueba
- De pre-producción
- De producción

CASO DE PRUEBA: set de condiciones variables bajo las cuales un tester determinará si el SW está funcionando correctamente o no. Una buena definición de CP nos ayuda a reproducir defectos.

Descubrir errores en forma completa con el mínimo esfuerzo y tiempo. Deriva desde el código, especificaciones de programación, requerimientos, info de relevamiento, documentos de cliente.

Condiciones de prueba: es la reacción esperada de un sist frente a un estímulo particular, constituido por las distintas entradas.

CASO DE PRUEBA → 3 PARTES

- objetivo: la exact del sist a comprobar
- datos de entrada y ambiente: datos a introducir en el sist.
- comportamiento esperado: salida esperada según los requerimientos del mismo

Estrategia → Caja Blanca / Caja Negra

~~**CAJA NEGRA:** Basado en Especificaciones Basado en la experiencia~~

Ciclo de Pruebas: abarca la ejecución de la totalidad de casos de prueba establecidos aplicados a una misma versión del sist a probar.

Regresión: al concluir un ciclo de pruebas y reemplazarse la versión del sist sometido al mismo, debe realizarse una verificación total de la nueva versión a fin de prevenir la introducción de nuevos defectos.

PROCESO DEL TESTING

- Planificación y control: verificar que se entiendan las metas y objetivos del cliente y stakeholders. Construcción del test plan. Riesgos y objetivos del testing, estrategia, recursos, criterios de aceptación. Control → revisar los resultados del testing, test coverage, tomar decisiones.
- Identificación y especificación: revisión de la base de las pruebas, identificar los datos necesarios, diseño del entorno de prueba, diseño y priorización de los casos de las pruebas, etc.
- Ejecución: crear los datos de prueba, desarrollar y dar prioridad a nuestros casos de prueba, automatizar lo necesario, implementar y verificar el ambiente, ejecutar los casos de prueba, comparar los resultados reales con los esperados.
- Evaluación y reportes: evaluar los criterios de aceptación, reportes de los resultados, lessons learned.

Verificación

Estamos construyendo el sist correctamente?

(vs)

Validación

Estamos constr. el sist correcto?

El testing exhaustivo es imposible. Decidir cuanto testing es suficiente depende de la evaluación del nivel de riesgo, costos asociados al proyecto.
Criterio de aceptación.

Smoke Test

Testing Funcional

Testing No funcional

Pruebas de interfaz de usuarios

Pruebas de performance

Pruebas de configuración

Tipos de pruebas

LEAN - KANBAN

Principios Lean

- eliminar desperdicio
- diferir compromisos
- embeber la integridad conceptual
- ver el todo
- dar poder al equipo
- simplificar aprendizaje
- entregar lo antes posible

Desperdicio en servicios basados en conocimiento

- trabajo parcialmente terminado
- características extra
- proceso extra
- cambio de tareas, transferencia de conocimiento
- espera/demora
- cambio de contexto
- defectos
- talento no utilizado

KANBAN: método para definir, gestionar y mejorar servicios que entregan trabajo del conocimiento.

Principios de Kanban

- | | | | |
|-----------------------------|---|---|------------------------|
| <u>Gestión de Cambios</u> | { | Comenzar con lo que hacen ahora. | Entender los procesos. |
| | | Buscar la mejora y el cambio evolutivo. | Respetar los roles. |
| | | Fomentar liderazgo en todos los niveles. | |
| <u>Entrega de Servicios</u> | { | Comprender y enfocarse en cumplir las necesidades del cliente. | |
| | | Gestionar el trabajo - los trabajadores se autoorganizan. | |
| | | Revisar la red de servicios para mejorar los resultados entregados. | |

PRÁCTICAS GUALES DE KANBAN

- visualizar
- gestionar el flujo
- hacer las políticas explícitas
- limitar el trabajo en curso
- establecer ciclos de retroalimentación
- mejorar colaborativamente, evolucionar experimentalmente

Políticas ejemplos:

Color de tarjeta, WIP, prioridad, forma de trabajo, cuándo se entrega, etc

KANBAN: MÉTRICAS

Lead Time → ① desde que se pide algo hasta que se entrega. Ritmo de entrega.

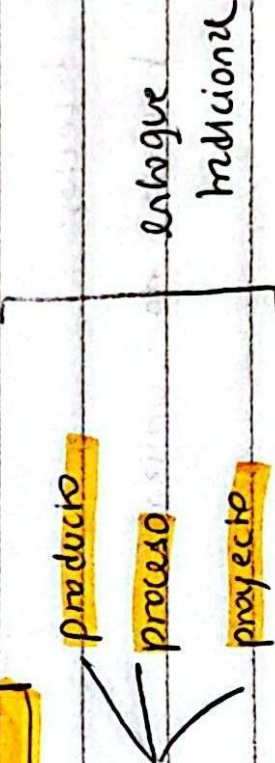
Cycle Time → ② desde que un item de trabajo se inicia y se termina. " de terminación.

Touch Time → ③ es el total el item ha realmente trabajado (no se cuentan las columnas como bloqueado).

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

$$\% \text{ eficiencia del proceso} = \frac{\text{Touch Time}}{\text{Cycle Time}} \quad !!!$$

MÉTRICAS EN LOS DIFERENTES ENFOQUES

El dominio de la métrica se divide en 

Métrica básica por un proyecto - enfoque tradicional

- tamaño del producto
- esfuerzo
- tiempo
- defectos

Métrica en ambiente ágil medir la velocidad y nada más

el SW funcionando es la principal medida de progreso

- Velocidad: cont de story points que el equipo termina en un sprint.

Se usa para predecir la capacidad del equipo.

- Capacidad: cuantos story points va a poder completar el equipo en el próximo sprint. Estimación.

- RTF (Running Tests Feature): cont de características del SW que han

sido implementadas y pueden de manera exitosa. Obtener una visión general del avance



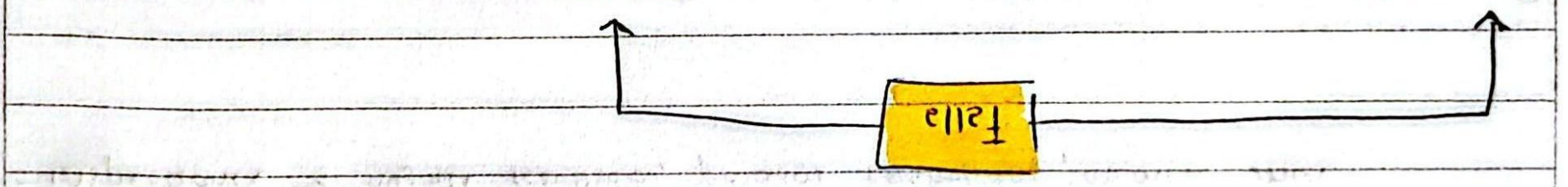
- Principios:**
- la prevención es mejor que la cura
 - vivir es más efectivo que eliminar
 - la remediación es más efectiva que eliminar
 - priorizar lo rentable
 - olvidar de la perfección (no se puede conseguir)
 - pensar a pensar en lugar de dar pensado.

Errores: originados en el producto

Mayor: condición que puede causar una falla operacional o producir un resultado inesperado durante la ejecución de la operación especificada.

Menor: condición que no es deseable en el producto pero que no puede ocasionar una falla operacional.

Cosméticos: error de tipo, ortografía, errata menor que no se vea como falla.



Revisiones Técnicas

- Verificación y Validación
- correctamente
- correcto

Calidad en el SW

Método para producto

Turno: líneas de código, requerimientos (cu, fecha, hora, story point)

Defectos: cobertura, defectos por severidad, densidad de defectos.

Revisiones Técnicas: proceso de V&V estático. Principal objetivo detectar defectos y corregirlos en las etapas tempranas de desarrollo.

No requieren que el programa se ejecute, motiva a realizar un mejor trabajo.

Ventaja: pueden descubrirse muchos errores, pueden inspeccionarse versiones incompletas, pueden considerarse otro atributo de calidad.

Desventajas: es difícil introducir las inspecciones formales. Sobrecarga al inicio los costos y conducen a un ahorro sólo después de que los equipos adquieren experiencia en su uso. Requieren tiempo.

Métodos

- walkthroughs → pocas métricas, no hay control de proceso
- inspecciones → mejores resultados, procesos controlados, métrica útil